

Complete Guide to Deploying Flutter Projects

From source code to live app fast, clean, and fully working.

Table of Contents

1. Introduction
 2. Requirements Checklist
 3. Setting Up Your Development Machine
 4. Running the Project Locally
 5. Building a Flutter Mobile App
 6. Deploying to Android (APK, AAB, Play Store)
 7. Deploying to iOS (IPA, App Store)
 8. Deploying Flutter Web Projects
 9. Managing Environment Configs
 10. Final Optimization & Publishing Tips
-

1. Introduction

Flutter lets you build cross-platform mobile (and web) apps from a single codebase. If you already have the project source code, this guide walks you through the steps to get it built and deployed.

Whether you're targeting Android, iOS, or Web — this guide tells you exactly what to do.

2. Requirements Checklist

Tool	Required Version
Flutter SDK	3.0+
Dart SDK	Included w/Flutter
Android Studio	Latest
Xcode (for iOS)	Latest (Mac only)
VS Code (Optional)	Recommended

3. Setting Up Your Development Machine

Install Flutter

- On macOS/Linux:

bash

CopyEdit

```
git clone https://github.com/flutter/flutter.git -b stable
export PATH="$PATH:`pwd`/flutter/bin"
flutter doctor
```

- On Windows:
Download and extract Flutter zip from <https://flutter.dev>, then run `flutter doctor` in PowerShell.

Check Setup

bash

CopyEdit

```
flutter doctor
```

Fix any red **X** errors before moving forward.

4. Running the Project Locally

Navigate to the project folder:

```
bash
CopyEdit
cd your_flutter_project
flutter pub get
flutter run
```

If you have a connected device or emulator, this will launch the app.

5. Building a Flutter Mobile App

 **Build APK (for Android):**

```
bash
CopyEdit
flutter build apk --release
```

APK will be generated at:

```
swift
CopyEdit
build/app/outputs/flutter-apk/app-release.apk
```

 **Build AAB (for Play Store):**

```
bash
CopyEdit
flutter build appbundle --release
```

AAB file will be here:

```
arduino
CopyEdit
build/app/outputs/bundle/release/app-release.aab
```

6. 📱 Deploying to Android (Google Play)

🔑 Step 1: Generate Keystore (If Not Yet Created)

bash

CopyEdit

```
keytool -genkey -v -keystore ~/my-key.jks -keyalg RSA -keysize 2048  
-validity 10000 -alias my-key-alias
```

🔧 Step 2: Add to **android/key.properties**

properties

CopyEdit

```
storePassword=your_store_password  
keyPassword=your_key_password  
keyAlias=my-key-alias  
storeFile=/Users/you/my-key.jks
```

🚀 Step 3: Upload AAB to Google Play Console

1. Go to <https://play.google.com/console>
2. Create your app → Upload **.aab** file under "Release" section
3. Complete policy forms & app details
4. Submit for review

7. 🍏 Deploying to iOS (App Store)

⚠️ Requires macOS and Xcode.

📱 Step 1: Build for iOS

bash

CopyEdit

```
flutter build ios --release
```

Step 2: Open in Xcode

bash

CopyEdit

```
open ios/Runner.xcworkspace
```

- Set signing & capabilities (Apple Developer Account required)
- Set Bundle ID and version



Step 3: Archive & Upload

1. In Xcode: Product → Archive
2. Then upload to App Store using Organizer
3. Go to App Store Connect and submit the app for review

8. Deploying Flutter Web Projects

Flutter can compile to a full static HTML/CSS/JS site.



Build:

bash

CopyEdit

```
flutter build web
```

This generates your web app in:

bash

CopyEdit

```
build/web/
```



Hosting Options:

Platform	How to Deploy
Firebase	<code>firebase deploy</code> (after <code>firebase init</code>)
Vercel	Connect repo, auto-deploys from <code>build/web</code>
Netlify	Drag <code>build/web</code> into the UI or CLI deploy
cPanel/Share d	Upload contents of <code>build/web</code> to <code>public_html</code>
VPS	Serve with Nginx (like a static site)

9. Managing Environment Configs

Flutter doesn't support `.env` natively, but you can use packages like:

`flutter_dotenv`

1. Add to `pubspec.yaml`:

```
yaml
CopyEdit
dependencies:
  flutter_dotenv: ^5.0.2
```

2. Create `.env` file:

```
ini
CopyEdit
API_URL=https://api.example.com
```

3. Load in `main.dart`:

```
dart
```

CopyEdit

```
import 'package:flutter_dotenv/flutter_dotenv.dart';
```

```
Future<void> main() async {  
  await dotenv.load();  
  runApp(MyApp());  
}
```

10. Final Optimization & Publishing Tips

-  Minify assets using `flutter build --release`
 -  Use HTTPS for API calls (avoid localhost in production)
 -  Set app version and build number in `pubspec.yaml`
 -  Use ProGuard for Android to obfuscate and reduce APK size
 -  Test on real devices for both platforms before launch
-

That's It — You're Deployed

Whether it's Android, iOS, or Web, your Flutter app should now be up and running. Stick to this flow, and you won't hit common roadblocks.